University of St. Gallen

School of Management, Economics, Law, Social Sciences, International Affairs, and Computer Science

---

Anticipating Returns in Luxury Fashion: A Dior Case Study in Predictive Modeling

Youri Gregorc

Alexandru Puspurica

Pavle Licina

Data Science Fundamentals Final Project

University of St. Gallen

Workshop Fundamentals of Data Science

Dr. Lyudmila Grigoryeva

Dr. Johannes Binswanger

29 November 2025

Table of Contents

1 Introduction

Predicting returns is important for any company for a multitude of reasons: they create reverse-logistics costs, refurbishment costs, require repackaging, and generate markdown losses all of which are necessary to properly plan in an operational budget. It is especially important for luxury brands for whom comparatively an individual sale represents a larger portion of overall sales than market-mass brands. As a result, luxury brands must build predictive models to estimate sales and returns for proper budgeting and operational resource alignment. Such models are normally based on historical time series data and incorporate trends, seasonality, and cyclical components. In our analysis we seek to construct an effective model to predict whether or not a product will be returned using probabilistic classification machine learning methods trained on one month historical data (Jan 2025) from Dior instead of multi year forecasting. We will begin by cleaning the data, then fitting different models, next the hyperparaters and thresholds will be tuned, and finally the models will be tested on our test data. It is also important to note that this classification type problem could only realistically be accurate for the month of January because of the high percentage of returns relative to sales (30% returns) even with a 30% return rate there is still a class imbalance which is accounted for by turning the thresholds. The high return rate in January can be explained by the higher amount of sales in December leading into the holiday season and Dior's 30 day return policy.

2 Preprocessing

The objective of our preprocessing was to transform the raw Dior January 2025 U.S. order dataset into a clean, structured, machine learning ready dataset suitable for return prediction modeling and descriptive analysis. The raw data contained mixed datatypes, missing values, categorical noise, redundant columns, and information leakage risks. The first step (i) taken was standardizing the column names and selecting only customer relevant features for this reason *latest status date local* and *fulfilment statu*s were dropped. There were also a few columns which were only filled when an order was classified as a return or vice versa. For this reason (to prevent data leakage) *is csc gift* and *shipping method* were dropped (ii) . Next fields not used in the US were dropped (iii): *emptygiftnote, mensualized status, isdeliverywaitandtry, has video gift message, sub-region.*

There were some features which included time series data which needed to be modified for use in a classification (iv), these columns included c*reated status date local* and *timespan booked_shipped order*. We thought it would be important to incorporate this time series data into our predictions because it could have an impact on return rates; Sometimes people make rash purchases they later regret during nighttime

hours which could be reelected in the data. We took the time the order was placed which included the full date time data down to the second and extracted the hourly information and classified this into a boolean true if the order was placed between 10pm and 5am, creating a column called *is_night_order*. For the *timespan book shipped orde*r which describes the days between which an order is placed and shipped there were 100s of individual values. For the sake of computing we categorized these points into 5 sections based on eyeballed relative distributions and rounding conventions. The classifications we created were <10 days, 10–20 days, 20–50 days, 50–100 days, >100 days. These classifications were then turned into bool columns.

We then focused on the return feature cleaning which was quite simple. Return number was converted into the boolean feature *is_return* (v), where presence of a return number indicates a return. This acted as our classifier or target variable. Columns describing why or how the return occurred were dropped because they occur after a customer decides to return an item these include the columns (vi): *return status, return reason en, sub-return reason en, return type, return address – state.* Dropping these columns also prevents leakage and ensures only pre-return information influences the model. The following features we cleaned dealt with product data information (vii). Many product sub-categories were duplicated due to inconsistent text formatting (extra spaces, inconsistent capitalization). There were also a lot of observations missing a subcategory so rows missing a subcategory were filled with their product category value as a fallback to increase the number of kept observations in the dataset. Because of the inconsistent categorical data for products we decided to make our own grouping (to see how different subcategories were grouped look at lines 204-306 in the preprocessing code) which we split into 8 groups: *Apparel (Clothing), Footwear, Bags & SLG, Jewellery & Timepieces, Accessories, Baby & Kids, Home & Lifestyle, Sports & Innovative*. A full set of boolean dummy variables was later created for these grouped subcategories. There were also a lot of sales and different quantity metrics which were modified (viii). Quantity was converted to its absolute value to avoid leakage (negative quantities indicated returns) and all observations with missing quantities were dropped. Because of the large number of unique entries we decided to make bands to save on computational power *sales incl. taxes (local)* was transformed into absolute value and binned into rational spending tiers (ix): 0–500, 500–2,000, 2,000–4,000, 4,000–6,000, 6,000–10,000, 10,000–20,000, 20,000–50,000, 50,000+. Dummy variables were created for each band and the original sales variable was removed. In the final step we converted the remaining categories to dummies and did a little bit of work removing the final missing or weird values (x) including rows with corrupted gender or state values ("****") and 4 other observations with missing data. In the end we had a clean data set with 149164 observations and 114 features all with boolean values. Before fitting the models on the data we created testing and training sets, stratified cross validation folds, and checked the

correlation to see if there was a final data leakage. This revealed an almost perfect negative correlation between timespan >10 days and *is_return* so we dropped this column as well as one column per categorical group to avoid the dummy variable trap (ex: one US state from shipping).

3 Model Creation and Evaluation Criteria

We decided to use eight different models to identify the best classifier for predicting whether an order would be accepted or returned. The models we selected are a majority-class predictor, Naive Bayes, logistic regression (with Lasso and Ridge regularization), random forests, boosting, and neural networks. Because the data showed a substantial class imbalance (around 70/30), we evaluated our models using both accuracy and F1-score. Accuracy measures the model's overall ability to correctly classify both classes (true positives and true negatives), while the F1-score evaluates the model's ability to detect as many positives as possible (recall) while keeping the number of false positives low (precision).

In this project, we first work with the global dataset. For each of the eight models, we tune the hyperparameters and the thresholds that separately maximize accuracy and F1-score. We then split the data into eight sub-categories: *Apparel (Clothing), Footwear, Bags & SLG, Jewellery & Timepieces, Accessories, Baby & Kids, Home & Lifestyle, Sports & Innovative*, which are trained exclusively to maximize the F1-score, as it is a more appropriate metric than accuracy for imbalanced datasets because it reflects the model's ability to correctly identify the minority class, rather than being inflated by the majority class. Indeed, as we observed in this dataset (see results section), a classifier that always predicts the most common class can achieve very high accuracy in an unbalanced dataset, even while being completely unable to distinguish between the two categories.

4 Hyperparameter Tuning and Model Threshold Adjustment

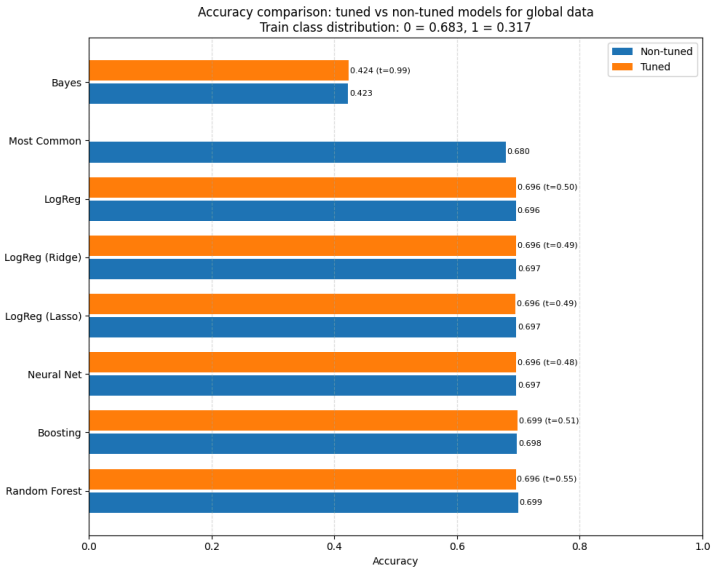
Tuning hyperparameters is commonly done to improve model performance and was a focus point in the bootcamp. Threshold adjustment, however, is something we were unaware of and helped greatly improve the performance of our models (see results). While classification models default to a threshold of 0.5, this value is rarely optimal in practice when you have unbalanced data. Unbalanced data creates a natural bias toward the class that minimizes the loss, the majority class, leading to overprediction compared to the other class. Adjusting the threshold allows us to counter this phenomenon, improving the model's ability to detect the minority class (as measured by the F1-score). This aspect of model optimization was not initially emphasized in our training, but ultimately proved to be one of the most impactful improvements in our results. We implemented threshold adjustment in three distinct ways:

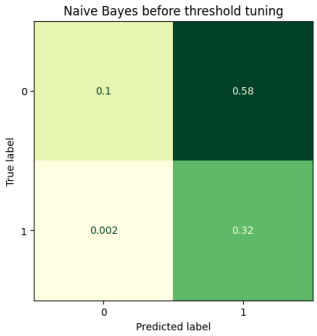
Method 1: We used, for the Logistic Regression and Naive Bayes models, a self-built threshold optimization function that looks on the training set for the threshold that leads to the best results.

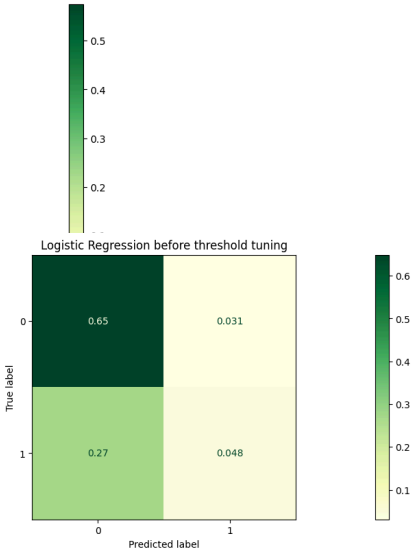
Method 2: Using TunedThresholdClassifierCV, we automatized and perfectionned this search using cross validation. We used this method for Boosting models and Neural Networks, where a cross-validated procedure helped stabilize threshold selection.

Method 3: Unlike the other approaches, which adjusted the threshold only after model training, this method wraps the model (in our case, the Random Forest) inside a custom estimator in order to include the threshold as a tunable hyperparameter. By exposing the threshold within the Randomized Search Cross-Validation, the model simultaneously optimizes its hyperparameters and its decision threshold. This integrated approach proved particularly effective for the Random Forest model.

5 Results for The Global Data

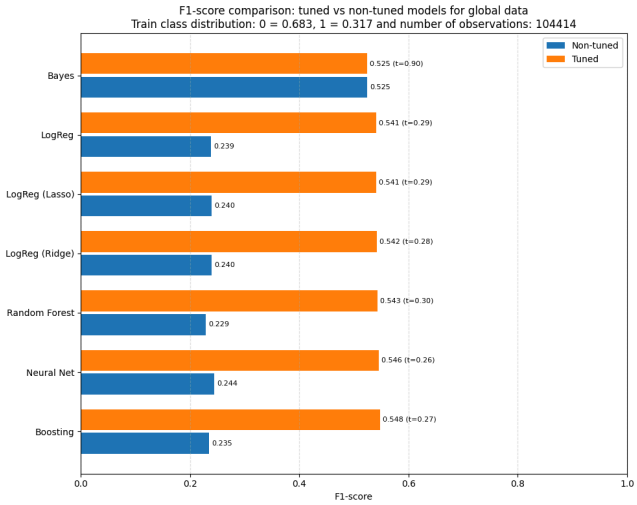


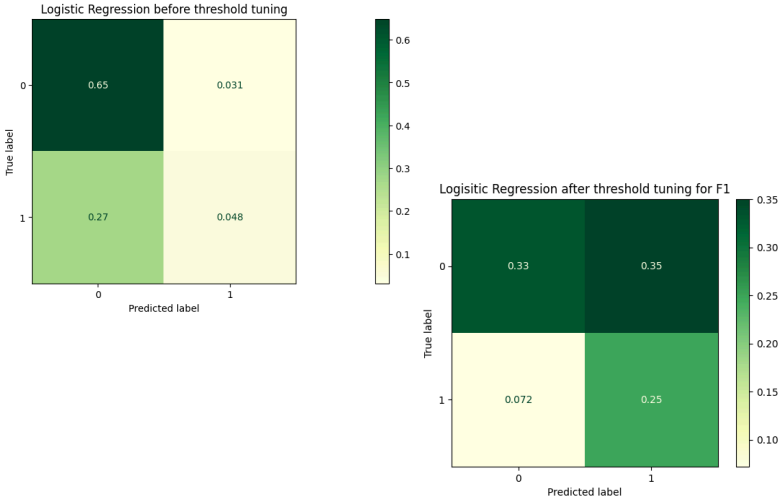




We can see for the accuracy that Bayes estimator performs very poorly while all the other classes are more or less equal in performance with both tuned and untuned parameters. The classifier which always predicts the most common class seems thus to be the best alternative. The poor performance of the Naive Bayes estimator can be explained by two fundamental issues. First, many features in the dataset (e.g., US states) are highly correlated, which violates the Naive Bayes assumption of conditional independence between features. Second, because the features are boolean, the assumption that they follow a normal distribution is also violated, making (Gaussian) Naive Bayes an inappropriate (but interesting) model for this type of data. The fact that its optimal threshold is 0.99 can be explained by the behavior of

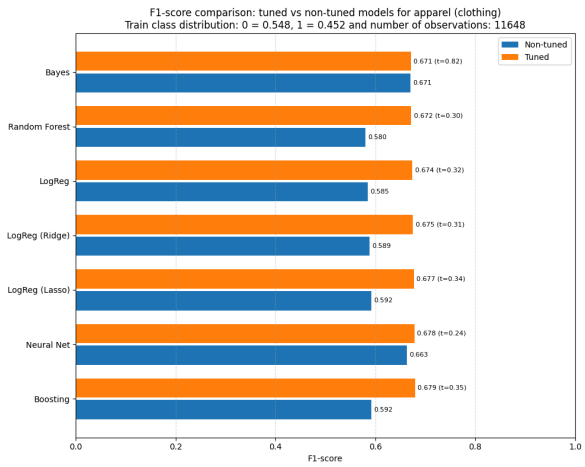
Naive Bayes: since it almost always predicts class 1, the threshold must be set extremely high to reduce the extraordinarily large number of false positives (see Naive Bayes confusion matrix before threshold tuning). For the other models, the optimal accuracy threshold remains close to 0.5. This can be explained by the class imbalance: lowering the threshold would produce more false positives than true positives, while increasing it would not improve performance, as the model already predicts most true negatives (see Logistic Regression confusion matrix before threshold tuning).

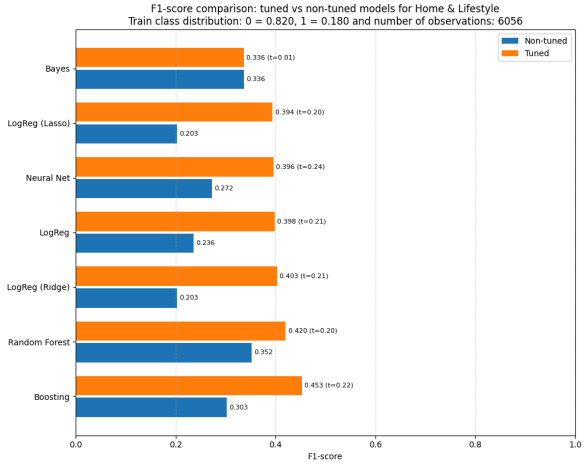




We can see that for the F1-score the models perform similarly: in their tuned versions, each achieves an F1-score between 0.525 and 0.548. In spite of being marginally more performant, there seems to be no need to use a more complex model than a simple Logistic Regression. To maximize the F1-score threshold tuning is essential to obtain higher results; threshold tuning enables some models to double their F1-score. For most models, the optimal threshold is drastically reduced between 0.26 and 0.30. This can be explained by the fact that lowering the threshold increases the number of class 1 observations detected (recall), but must eventually stabilize to prevent generating too many false positives (precision) (see Logistic Regression confusion matrix before threshold tuning and after threshold tuning). In contrast, Naive Bayes selects a very high optimal threshold. This can be explained by the fact that, unlike the other models, Naive Bayes already classifies nearly all true positives. To minimize the large number of false positives and thereby improve its precision, the threshold must be increased.

6 Results for Subcategories and Conclusion





These two sub-categories are representative of model performance across all eight groups. From them, we can draw four main conclusions applicable to our dataset. First, the larger the number of observations, the better the models can train, and the higher their performance tends to be. Second, when the dataset is balanced, threshold tuning becomes much less necessary. This is because the dataset does not push the model toward predicting class 0 (no majority-class bias). Moreover, overall performance is higher, as the model receives enough examples of both classes to learn them properly.

Third, as a consequence of the previous point, in an unbalanced dataset threshold tuning becomes essential in order to obtain meaningful results and substantially improve performance. Finally, the results show that there is no single model that performs best across all cases and performance does not depend solely on model complexity: a simple logistic regression model can perform better than a Random Forest (see F1-score comparison for apparel (clothing)).

List of Aids

Tool	Usage	Affected Section
ChatGPT	Correct the code; help design the graphs that present the results; help to find the function that wraps the Random Forest inside a custom estimator in order to include the threshold as a tunable hyperparameter; correct writing and syntactic mistakes in the report	Report/Code

Declaration of Authorship

"I hereby declare,

- that I have written this thesis independently,
- that I have written the thesis using only the aids specified in the index;
- that all parts of the thesis produced with the help of aids have been precisely declared;
- that I have mentioned all sources used and cited them correctly according to established academic citation rules;
- that I have acquired all immaterial rights to any materials I may have used, such as images or graphics, or that these materials were created by me;
- that the topic, the thesis or parts of it have not already been the object of any work or examination of another course, unless this has been expressly agreed with the faculty member in advance and is stated as such in the thesis;
- that I am aware of the legal provisions regarding the publication and dissemination of parts or the entire thesis and that I comply with them accordingly;
- that I am aware that my thesis can be electronically checked for plagiarism and for third-party authorship of human or technical origin and that I hereby grant the University of St.Gallen the copyright according to the Examination Regulations as far as it is necessary for the administrative actions;
- that I am aware that the University will prosecute a violation of this Declaration of Authorship and that disciplinary as well as criminal consequences may result, which may lead to expulsion from the University or to the withdrawal of my title."

By submitting this thesis, I confirm through my conclusive action that I am submitting the Declaration of Authorship, that I have read and understood it, and that it is true.

Youri Gregorc, Alexandru Puspurica, Pavle Licina

(13642 characters)